UI extensibility

Fetching data for UI extensions

Learn how to use the `hubspot.fetch` API to fetch data for UI extensions, such as app cards, home pages, and settings pages.

UI extensions can be built to request data from an external source using the `hubspot.fetch()` API. At a high level, to fetch data using this method, you'll need to:

- Provide a REST-based back-end service to handle requests.
- Include the request URLs as `permittedUrls` in the **app's configuration**.

Below, learn more about using `hubspot.fetch()` to request data from a UI extension.

Limits

Requests made with `hubspot.fetch()` are subject to the following limits:

- Each app is allowed up to 20 concurrent requests per account that it's installed in. Additional requests will be rejected with the status code `429`, and can be retried after a delay.
- Each request has a maximum timeout of 15 seconds. Both request and response payloads are limited to 1MB. You can specify a lower timeout per request using the `timeout` field. Request duration time includes the time required to establish an HTTP connection. HubSpot will automatically retry a request once if there are issues establishing a connection, or if the request fails with a `5XX` status code within the 15 second window.

If you're familiar with using the **browser Fetch API**, note that `hubspot.fetch()` is not a one-to-one replacement. Key differences between the two APIs are listed below.

HubSpot Developers

›

Cache control	Supported	Not supported
Timeout	Browser default	15 seconds max
Payload size	Large	1MB max
URL requirements	Any URL	Must be in `permittedUrls`

Method

The method contract for `hubspot.fetch()` is as follows:

```
import { hubspot } from '@hubspot/ui-extensions';

interface Options {
  method?: 'GET' | 'PUT' | 'POST' | 'DELETE' | 'PATCH';
  timeout?: number;
  body?: {
    [key: string  | number]: unknown;
  }
}

hubspot.fetch(resource: string | URL): Promise<Response>
hubspot.fetch(resource: string, options?: Options): Promise<Response>
```

Parameter	Type	Description
`method`	String	The method to use.
`timeout`	Number	Time in milliseconds to allow for the request to complete before timing out. Timeout will occur when the back-end request duration exceeds this value or 15 seconds—whichever is smaller. Request duration time includes the time required to establish an HTTP connection.HubSpot will retry a request once if there are issues establishing a connection, or if the request fails with a `5XX` status code within the 15 second window.

⚠ `hubspot.fetch()` does not support request and response headers. The HubSpot signature must be used to perform authentication/authorization in your back-end. For security reasons, you should not store secrets in your React code to communicate with third-party back-ends. Instead, use your own back-end to fetch data from other third-party APIs after **validating the HubSpot signature** on the request.

Specify URLs to fetch from

To make calls to your backend or a third-party service, update the app's `*-hsmeta.json` **configuration file** to include the URLs you'll be requesting. The URLs must be specified in the `fetch` array of `permittedUrls` .

Note that fetch URLs must be valid HTTPS URLs and cannot be `localhost` . If you want to send requests to a locally running back-end, learn how to **proxy requests**.

```
"permittedUrls": {
    "fetch": ["https://api.example.com/api/data/"],
    "img": [],
    "iframe": []
}
```

⚠ When running the local development server, any `hubspot.fetch()` request will still go to your hosted back-end via a HubSpot-managed data fetch service. If you need to update the allowlist while running the dev server, you'll need to run `hs project upload` for the change to take effect.

Headers and metadata

To ensure that the requests hitting your back-end are coming from HubSpot, several headers are included in the request. You can use these headers, along with the incoming request fields, to verify the signature of the request. Learn more about

.

HubSpot will always populate headers related to request signing and also allow you to pass an `Authorization` header from `hubspot.fetch()` . See the

HubSpot also automatically adds the following query parameters to each request to supply metadata: `userId` , `portalId` , `userEmail` , and `appId` . As the request URL is hashed as part of the signature header, this will help you securely retrieve the identity of the user making requests to your back-end.

> ⚠ If you're not seeing appended metadata passed with `hubspot.fetch()` requests, check whether you have a `local.json` file that's currently rerouting requests via a proxy. If so, disable this local data fetch feature by renaming the file to `local.json.bak` and restarting the development server.

Proxying requests to a locally running back-end

If you have a locally running back-end, you can set up a proxy to remap `hubspot.fetch()` requests made during local development. This proxy is configured through a `local.json` file in your project, and will prevent requests from being routed through HubSpot's data fetch service.

To proxy requests to a locally running back-end:

Create a `local.json` file in the same directory as your `public-app.json` file. In this file, define a proxy that remaps requests made using `hubspot.fetch()` . This mapping will only happen for the locally running extension. You can include multiple proxy key-value pairs in the `proxy` object.

```
{
  "proxy": {
    "https://example.com": "http://localhost:8080"
  }
}
```

> ⚠ Each proxy URL must be a valid URL and use HTTPS.

HubSpot Developers

›

With your project uploaded, run `hs project dev` to start the development server. The CLI should confirm that it has detected your proxy.

```
[BETA] HubSpot projects local development

Running joe-test-public-app locally on ProdDevTest [developer test account] (        ), waiting for changes ...
View project in HubSpot: https://app.hubspot.com/developer-projects/        /project/joe-test-public-app

Press 'q' to stop the local dev server
--------------------------------------------------

[INFO] Proxy config discovered, enabling local proxy mode
```

When you request the mapped URL using `hubspot.fetch()`, the CLI will confirm the remapping.

```
[BETA] HubSpot projects local development

Running joe-test-public-app locally on ProdDevTest [developer test account] (        ), waiting for changes ...
View project in HubSpot: https://app.hubspot.com/developer-projects/        /project/joe-test-public-app

Press 'q' to stop the local dev server
--------------------------------------------------

[INFO] Proxy config discovered, enabling local proxy mode
[INFO] Serving app functions locally (platform version 2023.2)
[INFO] Running extension 'Hello World' from app 'Joe's super cool test app'
[INFO] Browser connected and listening for bundle updates
[INFO] Request to https://run.mocky.io/                                    started, method=GET, correlationId=36e70a5d-3ece-447d-8a98-44d37dac428c
[INFO] Mapping https://run.mocky.io/                                    -> https://run.example.io/                                    , correlationId=36e70a5d-3ece-447d-8a98-44d37dac428c
[INFO] Request completed, correlationId=36e70a5d-3ece-447d-8a98-44d37dac428c
```

Request signatures during proxied local development

By default, when proxying requests during local development, requests will not be signed or include metadata in query parameters. However, if you want to introduce request signing into the local development process, you can inject the `CLIENT_SECRET` environment variable into the local development process.

Injecting CLIENT_SECRET

After setting up your `local.json` file to proxy specific domains, you can inject the `CLIENT_SECRET` variable when starting the local development server by prepending the `hs project dev` command with the variable:

```
CLIENT_SECRET="abc123" hs project dev
```

HubSpot Developers

›

JavaScript Python

```
CLIENT_SECRET="abc123" node my-app.js
// and also
CLIENT_SECRET="abc123" npm run dev
```

And to access the `CLIENT_SECRET` variable:

```
console.log(process.env.CLIENT_SECRET);
```

Once you've finalized your request signing logic and have permanently added it to your back-end code, you'll need to inject the `CLIENT_SECRET` variable from your app into the `hs project dev` command permanently. For ease of use, consider baking the variable into your start scripts for local development.

To validate HubSpot request signatures in your custom back-end, check out the . You can also use HubSpot's npm module to verify requests and sign them yourself. For example:

HubSpot Developers

```
const clientSecret = process.env.CLIENT_SECRET
const signatureV3 = req.header('X-HubSpot-Signature-v3');
const timestamp = req.header('X-HubSpot-Request-Timestamp');

// Reject the request if the timestamp is older than 5 minutes.
if (parseInt(timestamp, 10) < (Date.now() - 5 * 60 * 1000)) {
  throw Error('Bad request. Timestamp too old.')
}

const requestBody = req.body === undefined || req.body === null
  ? ''
  : req.body;

const validV3 = Signature.isValid({
  signatureVersion: 'v3',
  signature: signatureV3,
  method,
  clientSecret,
  requestBody,
  url,
  timestamp,
});

if (!validV3) {
  throw Error('Bad request. Invalid signature.')
}
```

hubspot.fetch examples

Below are examples of `hubspot.fetch` requests to illustrate promise chaining, async/await, and authorization header usage.

Promise chaining

HubSpot Developers

```
const Hello = ({ context }) => {
  useEffect(() => {
    let url = "https://run.mocky.io/v3/98c56581"; // replace with your own
    hubspot
      .fetch(url, {
        timeout: 2_000,
        method: "GET",
      })
      .then(response => {
        console.log("Server response:", response.status);
        response
          .json()
          .then(data => console.log(data))
          .catch(err => console.error("Failed to parse as json", err));
      })
      .catch(err => {
        console.error("Something went wrong", err);
      });
  }, []);

  return <Text>Hello world</Text>;
};
```

Async/await

HubSpot Developers

```
const Hello = ({ context }) => {
  useEffect(() => {
    const fetchData = async () => {
      let url = "https://run.mocky.io/v3/98c56581"; // replace with your own
      const response = await hubspot.fetch(url, {
        timeout: 2_000,
        method: "GET",
      });
      console.log("Server response:", response.status);
      try {
        const data = await response.json();
        console.log(data);
      } catch (err) {
        console.error("Failed to parse as json", err);
      }
    };
    fetchData().catch(err => console.error("Something went wrong", err));
  }, []);

  return <Text>Hello world</Text>;
};
```

Authorization header

You may return a short-lived authorization token from your back-end service after validating the HubSpot signature. You can then use this token to access other resources.

To get the access token from your back-end server in the UI extension:

```
.then((response: Response) => {
  response.json().then((data) => setAccessToken(data.accessToken));
})
```

To return a short-lived access token from your back-end server:

```
app.get("/get-access-token", (req, res) => {
  validateHubspotSignatureOrThrow(req);
  res.json({
    accessToken: generateShortLivedAccessToken(req.query.userEmail),
    expiryTime: addMinutes(currentTime, 10),
  });
});
```

To attach access tokens to other UI extension requests:

```
hubspot.fetch('https://www.oauth-enabled-api.com/', {
  timeout: 3000,
  method: 'GET',
  headers: {
    'Authorization': `Bearer ${accessToken}`
  }
}).then((response: Response) => {
  ...
})
```

Monitoring and logs

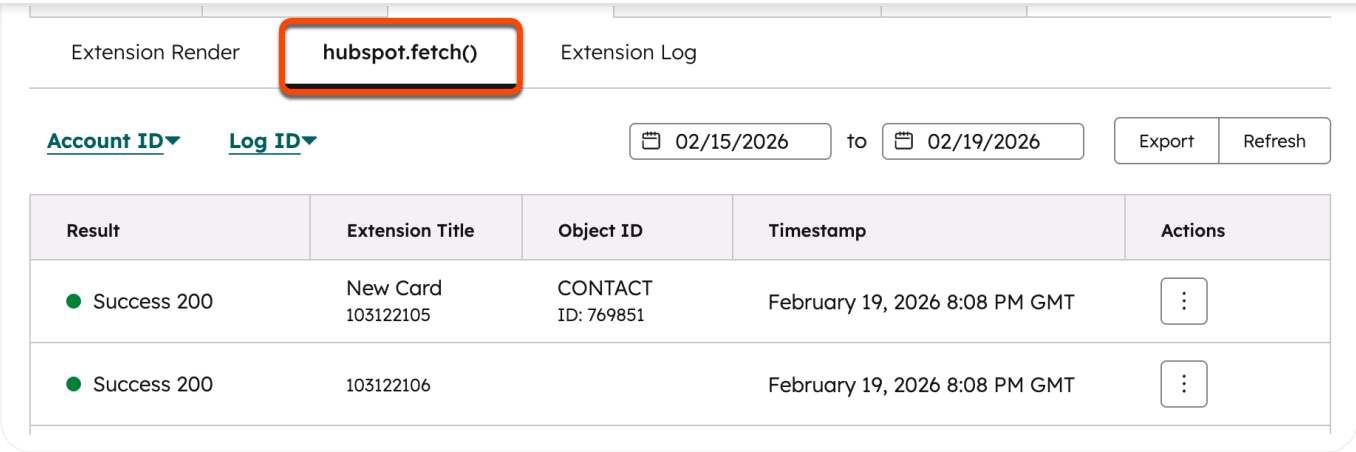
To monitor app card activity, you can view request logs in HubSpot:

In your HubSpot account, navigate to **Development**.

In the left sidebar, navigate to **Monitoring** > **Logs**.

On the monitoring page, click the **UI Extensions** tab.

HubSpot Developers



Last modified on February 19, 2026

Was this page helpful? Yes No

Logging and monitoring for UI extensions
Previous

UI extensions SDK reference
Next

HubSpot Developers
Copyright © 2026 HubSpot, Inc

Privacy Policy

Security

Website Accessibility

Manage Cookies